

Métodos & Propriedades do TestComplete

SetText vs Keys — Inserindo Valores

Ambas inserem texto em um campo, mas funcionam de forma diferente:

SetText

```
campo.SetText('12345');  
// "empurra" o valor inteiro de uma vez
```

- Rápido — ideal para campos simples.
- Pode não disparar eventos de digitação.
- Recomendado para IDs, códigos, senhas.

Keys

```
campo.Keys('12345');  
// simula cada tecla pressionada
```

- Dispara eventos de teclado (onChange, onKeyPress).
- Use quando o sistema valida a digitação.
- Necessário para campos com máscara.

Combinações especiais com Keys

```
campo.Keys('[Enter]'); // pressiona Enter  
campo.Keys('[Tab]');  // pressiona Tab  
campo.Keys('^a');      // Ctrl+A  
campo.Keys('[F4]');    // tecla F4  
campo.Keys('[P500]');  // pausa 500 ms
```

Referência: Teclas Especiais com Keys()

Código	Tecla	Código	Tecla
^	Ctrl	[Enter]	Enter
!	Shift	[Esc]	Escape
~	Alt	[Tab]	Tab
[BS]	Backspace	[Del]	Delete
[F1]...[F12]	F1 até F12	[Home]	Home
[End]	End	[Up][Down]	Setas
[Left][Right]	Setas	[PgUp][PgDn]	Page Up/Down
[Pnnn]	Pausa nnn ms	[Win]	Tecla Windows

```
// Exemplos práticos
Sys.Process('APP').VCLObject('Edit1').Keys('^a[Del]'); // seleciona tudo e apaga
Sys.Process('APP').VCLObject('BtnSalvar').Keys('[Enter]');
Sys.Process('APP').VCLObject('Edit1').Keys('[P300]12345'); // pausa 300ms antes
```

Guia Rápido — Todas as Propriedades de Estado

Referência completa — use este slide como consulta rápida durante o desenvolvimento dos scripts.

Propriedade	Tipo	Valores	Descrição / Quando usar
Enabled	Boolean	true / false	Campo aceita clique e digitação? false = cinza/bloqueado
Visible	Boolean	true / false	Prop. Visible do objeto = true no código do sistema
VisibleOnScreen	Boolean	true / false	Objeto realmente visível e clicável para o usuário 
Exists	Boolean	true / false	Objeto encontrado na hierarquia — não gera erro se false
wText	String	texto livre	Texto exibido: campo, label, botão, título de janela
wItemCnt	Integer	0..N	Quantidade de itens em ComboBox ou ListBox
Checked	Boolean	true / false	CheckBox ou RadioButton está marcado?
Focused	Boolean	true / false	Objeto possui o foco do teclado no momento?
Left / Top	Integer	pixels	Posição X e Y do objeto na tela
Width / Height	Integer	pixels	Largura e altura do objeto em pixels
Name	String	texto livre	Nome do componente conforme mapeamento no TC
FullName	String	path completo	Caminho completo do objeto na hierarquia

```
function VerificarObjeto(AObj: OleVariant; ANome: string): boolean;  
begin  
    result := false;  
    if not AObj.Exists then begin Log.Error('Não existe: ' + ANome); Exit; end;  
    if not AObj.Visible then Log.Error('Objeto não visível: ' + ANome); Exit;  
    if not AObj.Enabled then Log.Error('Objeto não habilitado: ' + ANome); Exit;  
    if not AObj.VisibleOnScreen then Log.Error('Objeto não visível na tela: ' + ANome); Exit;  
    result := true;  
end;
```

Enabled vs Visible vs VisibleOnScreen — Qual usar?

Enabled → Boolean

O objeto aceita interação do usuário?

true = pode clicar/digitar
false = aparece acinzentado

Não diz nada sobre visibilidade. Um campo pode estar enabled mas completamente oculto.

Visible → Boolean

A prop. Visible do componente está como true?

true = não está oculto por propriedade/CSS
false = está marcado oculto

Pode ser true mas estar fora da área visível da janela.

VisibleOnScreen → Boolean Mais seguro

O usuário realmente consegue ver e interagir?

true = visível E dentro da área clicável da tela
false = oculto, coberto ou fora da viewport

Use quando for clicar ou ler.

Tabela Comparativa — Combinações possíveis

Cenário	Exists	Visible	VisibleOnScreen	Enabled
Objeto normal e interativo	true	true	true	true
Objeto desabilitado (cinza)	true	true	true	false
Objeto oculto (Visible=false)	true	false	false	true
Objeto fora da viewport	true	true	false	true
Objeto inexistente	false	erro	erro	erro

Exists — Verificação Segura de Objetos

Use Exists sempre antes de acessar um objeto que pode não existir — evita exceções de runtime.

❌ Jeito Errado — Sem verificação

```
// PROBLEMA: se frmCadastro não abriu,  
// o TC lança exceção e para o teste!  
var frm: OleVariant;  
frm := Sys.Process('APP')  
        .VCLObject('frmCadastro');  
// Esta linha vai crashar:  
frm.VCLObject('btnOK').Click;
```

✅ Jeito Certo — Com Exists

```
// SEGURO: Exists retorna false  
// sem gerar erro  
var frm: OleVariant;  
frm := Sys.Process('APP')  
        .VCLObject('frmCadastro');  
if frm.Exists then  
    frm.VCLObject('btnOK').Click;  
else  
    Log.Warning('Janela não encontrada');
```



Nunca acesse propriedades de um objeto sem antes garantir que Exists = true. Acessar .Visible, .Enabled etc. em um objeto inexistente gera exceção imediata.

Padrão recomendado — ordem de verificação antes de interagir:

```
if not btn.Exists then begin Log.Error('Não encontrado'); Exit; end;  
if not btn.VisibleOnScreen then begin Log.Warning('Não visível'); Exit; end;  
if not btn.Enabled then begin Log.Warning('Desabilitado'); Exit; end;  
btn.Click; Log.Checkpoint('Clique OK!');
```

Propriedades de Conteúdo — wText, wItemCount, Checked, Focused

wText → String

Retorna o texto visível de qualquer componente: campo, label, botão, título de janela.

```
var txt: string;
txt := frm.VCLObject('lblStatus').wText;
if txt = 'Gravado com sucesso!' then
    Log.Checkpoint('Status OK')
else
    Log.Error('Status inesperado: ' + txt);
```

Checked → Boolean

Verifica se um CheckBox ou RadioButton está marcado (true) ou desmarcado (false).

```
var chk: OleVariant;
chk := frm.VCLObject('chkAtivo');
if chk.Checked then
    Log.Message('Ativo: SIM')
else
    Log.Message('Ativo: NÃO');
chk.ClickChecked; // alterna estado
```

wItemCount → Integer

Retorna a quantidade de itens em ComboBox, ListBox ou similares.

```
var qtd: integer;
var combo: OleVariant;
combo := frm.VCLObject('cboFilial');
qtd := combo.wItemCount;
if qtd > 0 then
    Log.Message('Itens: ' + IntToStr(qtd))
else
    Log.Error('ComboBox vazio!');
```

Focused → Boolean

Verifica se o componente está com o foco do teclado. Útil para validar tab-order.

```
campo.Keys(['Tab']);
var prox: OleVariant;
prox := frm.VCLObject('edtNome');
if prox.Focused then
    Log.Checkpoint('Tab-order OK');
```

WaitProperty — Sincronizar com o Estado do Objeto

WaitProperty aguarda que uma propriedade atinja o valor esperado dentro de um timeout. Muito mais inteligente que Delay().

Sintaxe: `objeto.WaitProperty(NomePropriedade, ValorEsperado, TempoLimiteMs)` | Retorna TRUE se atingido, FALSE se timeout.

```
// 1. Aguardar botão ficar habilitado
if btn.WaitProperty('Enabled', true, 5000) then
    btn.Click
else Log.Error('Botão não habilitou em 5 segundos');

// 2. Aguardar campo aparecer na tela
if campo.WaitProperty('VisibleOnScreen', true, 3000) then
    campo.SetText('001')
else Log.Error('Campo não ficou visível');

// 3. Aguardar mensagem de sucesso
if msg.WaitProperty('wText', 'Gravado com sucesso!', 8000) then
    Log.Checkpoint('Gravação confirmada na tela')
else Log.Error('Mensagem de sucesso não apareceu');

// 4. Aguardar checkbox marcado
if chkConfirmado.WaitProperty('Checked', true, 2000) then
    Log.Message('Checkbox marcado automaticamente pelo sistema');
```

 Prefira sempre `WaitProperty` a `Delay()` fixo — ele avança imediatamente quando a condição é atingida, sem desperdiçar tempo.

WaitProperty — Aguardar Estado de um Objeto

Use WaitProperty para sincronizar seu script com o tempo de resposta da aplicação.

```
// Sintaxe
objeto.WaitProperty(NomePropriedade, ValorEsperado, TempoLimiteMs)
// Retorna TRUE se atingido, FALSE se timeout

var btn: OleVariant;
btn := Sys.Process('APP').VCLObject('btnSalvar');

// 1. Aguarda o botão ficar habilitado por até 5 segundos
if btn.WaitProperty('Enabled', true, 5000) then
    btn.Click
else
    Log.Error('Botão Salvar não habilitou no tempo esperado');

// 2. Aguarda campo ficar visível
if campo.WaitProperty('VisibleOnScreen', true, 3000) then
    campo.SetText('12345');

// 3. Aguarda mensagem de sucesso aparecer
if msgLabel.WaitProperty('wText', 'Gravado com sucesso!', 8000) then
    Log.Checkpoint('Gravação confirmada na tela');
```



Dica: Prefira WaitProperty a Delay() — é mais inteligente e não desperdiça tempo.

Localizar Objetos: Find, FindChild e Exists

```
// FindChild - busca um filho com propriedades específicas
var grid: OleVariant;
grid := Sys.Process('APP').VCLObject('frmPrincipal')
        .FindChild('WndClass', 'TDBGrid', 5);

// FindChild com múltiplas propriedades
var btn: OleVariant;
btn := Sys.Process('APP').VCLObject('frmPrincipal')
        .FindChild(['WndClass', 'wText'], ['TButton', 'Salvar'], 5);

// Verificar se o objeto existe antes de usar
if grid.Exists then
    Log.Message('Grid encontrado: ' + grid.FullName)
else
    Log.Warning('Grid não encontrado na tela');

// WaitVCLObject - aguarda o objeto surgir
var frm: OleVariant;
frm := Sys.Process('APP').WaitVCLObject('frmCadastro', 5000);
```

- Exists retorna FALSE (não gera erro) — sempre mais seguro que VCLObject direto.
- O terceiro parâmetro de FindChild é a profundidade de busca (recomendado: 5).
- Use WaitVCLObject quando uma janela abre de forma assíncrona.

Melhorando o Log do Teste

Função	Quando usar
Log.Message('texto')	Mensagens informativas gerais
Log.Error('texto')	Falha crítica — marca o teste como FAILED
Log.Warning('texto')	Alerta — não falha o teste
Log.Checkpoint('texto')	Validação bem-sucedida
Log.Event('texto')	Evento relevante do fluxo
Log.Link('url','rótulo')	Adiciona link (ex: caso no Jira)
Log.Picture(objeto,'nome')	Captura screenshot do objeto
Log.LockEvents()	Suprime eventos de clique/teclado do log

```
Log.LockEvents(); // No início do script — elimina ruído de eventos no log
Log.Checkpoint('Pedido ' + numPed + ' gravado com sucesso — posição: L');
Log.Error('CONDVENDA divergente — esperado: 1, recebido: ' + condvenda);
Log.Link('https://jira.empresa.com/CT-99', 'CT-99 — Caso de Teste');
```

Debug — Depurando seu Script

F11

Executa linha por linha — entra dentro das funções chamadas.

F10

Executa linha por linha — NÃO entra dentro das funções.

F5

Continua a execução até a próxima marcação de debug ou fim.

Ctrl+F10

Mostra o valor da variável selecionada no momento.

Ctrl+F12

Abre a janela Evaluate — avalia qualquer expressão.

Como criar uma marcação de debug: clique na margem esquerda do editor ao lado da linha desejada — um círculo vermelho indica o breakpoint ativo.